SECURITY LAYER 1

Authenticator and transaction approval

This appendix adds the project research on PINs, Authenticator codes, and approval. It distinguishes the intended security concepts from what the current prototype actually implements.

Wallet PIN

The server stores a salted PBKDF2-HMAC-SHA256 verifier (600,000 iterations). The 4-digit demo PIN is checked by the Flask backend for server-backed actions.

Authenticator code

The customer enrolls a TOTP secret using a setup QR. Approval uses a six-digit code that changes every 30 seconds, together with the wallet PIN.

How approval works

1. **Enroll**
   Scan the setup QR with an Authenticator app and confirm a current code.

2. **Unlock**
   Enter the demo wallet PIN to unlock the customer device credential.

3. **Verify**
   Validate a six-digit TOTP code and reject a previously used time step.

4. **Continue**
   Only then continue to the simulated payment approval flow.

Implementation note

The source research described the Authenticator as a PIN-only check. The current app uses both the wallet PIN and a TOTP code; they are separate checks.

SECURITY LAYER 2

HTTPS and the local certificate

A CA certificate and a server certificate are different parts of a trust chain. The current files show a local development HTTPS setup for the Flask server.

LOCAL DEV CA
Trust anchor

->

SERVER CERTIFICATE
Hostname identity

->

BROWSER
Checks trust + hostname

Current implementation

The Flask app is configured with netbridge-lan-cert.pem and its matching private key. The certificate is a server certificate signed by a local mkcert development CA. Its subject alternative names cover localhost, 127.0.0.1, and the project LAN address.

Item	Role in NetBridge
Server certificate	Presented by Flask during HTTPS page and API connections.
Local CA certificate	Lets a client validate the development server certificate only if that CA is trusted on the client device.
TLS / HTTPS	Encrypts browser-to-server traffic and supports server identity checks when trust and hostname validation succeed.

What this does not mean

The local certificate does not create a direct TLS session between the customer phone and merchant PC. The face-to-face offline handoff uses a request QR and a typed confirmation code after both pages are cached. It has no network exchange at payment time.

Device trust

If a phone does not trust the local development CA, the browser may warn about the HTTPS certificate. Do not share the private key; do not treat a development CA as production trust.

SECURITY BOUNDARIES

Which connection is protected?

The offline handoff and far-recipient simulation use different paths. Certificate protection applies where a browser talks to the NetBridge server.

Activity	Connectivity	Security channel and boundary
Initial setup and enrollment	Server reachable	Browser uses HTTPS to Flask. Wallet PIN and Authenticator enrollment are checked by the app. The browser must trust the issuing CA.
Face-to-face payment after caching	No network needed during handoff	Customer scans the merchant request QR, approves locally, then shows a 10-character code for the merchant to type. The devices keep separate local records; TLS is not carrying this handoff.
Far-recipient payment while offline	Queued on sender device	The receiver has not received anything yet. The pending item remains on the sender device.
Far-recipient synchronization	Server reachable again	Sender syncs with Flask over HTTPS. The recipient can then see a simulated delivery through the server-backed view.

Main distinction

HTTPS protects the app-server connection. It does not turn the QR/manual-code exchange into a peer-to-peer network payment, nor does it synchronize device-local histories during the offline demonstration.

The customer cannot see the merchant acceptance without a return communication path. The far-recipient flow is deferred server synchronization, not offline settlement between two phones.

RESEARCH TAKEAWAYS

Current safeguards and next steps

The supporting research raises useful security questions. The notes below identify existing controls and separate them from production recommendations.

Area	Current prototype	Recommended next step
Wallet PIN	Salted PBKDF2-HMAC-SHA256 verifier; 4-digit demo value.	Keep rate limits. Replace demo PINs with stronger user-selected credentials before any real deployment.
TOTP secret	Server SQLite stores the secret in plaintext. The customer browser also stores an encrypted offline credential using PIN-derived AES-GCM.	Encrypt server-side secrets at rest with a managed key. Treat a compromised browser or weak PIN as a risk to offline data.
OTP replay	TOTP verification tracks time steps and rejects reuse; server enrollment has an attempt lockout.	Keep replay state atomic and test expiry, replay, wrong-code, and lockout behavior across reconnects.
TLS trust	Local mkcert development certificate serves the Flask site over HTTPS.	Protect the private key. For deployment, use an appropriate trusted certificate and managed renewal/trust distribution.
Offline records	Each disconnected device stores its own demo state. No global double-spend prevention.	Keep the simulation label. Real offline settlement needs a separate protocol and reconciliation design.

Presentation wording

Describe the face-to-face outcome as “payment recorded in the simulation.” Describe far payments as pending until server synchronization completes. Do not say that a bank transfer or real money settlement occurred.

Security status: educational prototype only. These controls are useful demonstrations, not a security certification or production payment guarantee.